CycleWatch — Project Documentation

CS 479 Final Project Author: Rafael Mejia Date: April 2026

Table of Contents

1. Project Overview
2. Technology Stack (Plain English)
3. Hardware & Sensors
4. Arduino Code Explained
5. How the Browser Talks to the Arduino
6. Data Flow — End to End
7. React Hooks (The Logic Layer)
8. Application State & Context
9. UI Components
10. The Emergency Call API
11. Settings & Local Storage
12. Constants & Thresholds
13. Key Code Examples

---

1. Project Overview

CycleWatch is a **real-time cyclist safety dashboard**. A set of physical sensors is mounted on a bike and wired to an ESP32 microcontroller. The ESP32 reads all the sensors every 200 ms and sends the data over USB as JSON text. A web app (running in the browser) reads that USB data and displays it in a live dashboard. If anything dangerous is detected — a car in the blind spot, a crash impact, or low blood oxygen — the app alerts the rider immediately and can automatically call an emergency contact.

In one sentence: sensors on the bike → ESP32 formats the data → USB serial cable → browser reads it → React updates the screen.

---

2. Technology Stack (Plain English)

What is React?

React is a JavaScript library made by Meta for building user interfaces. Instead of manually updating HTML when data changes, you describe **what the screen should look like given the current data**, and React figures out the minimum set of DOM updates needed. The core idea is the **component**: a reusable piece of UI (like a metric tile or a radar display) that owns its own logic and appearance.

```
// A React component is just a function that returns HTML-like JSX
function MetricTile({ label, value, unit }) {
  return (
    <div>
      <p>{label}</p>
      <span>{value}</span>
      {unit && <span>{unit}</span>}
    </div>
  );
}
```

```
// Use it anywhere like an HTML tag:
<MetricTile label="Speed" value={34} unit="km/h" />
```

React also provides **hooks** — functions that let components react to changing data over time. The most important ones used in this project are `useState` (store a value), `useEffect` (run code when a value changes), `useCallback` (cache a function so it is not recreated every render), and `useRef` (hold a value that does not trigger re-renders, like a timer ID).

What is Next.js?

Next.js is a framework built on top of React that adds a full project structure, a built-in web server, routing (each file in the `app/` folder automatically becomes a URL), and **API routes** (server-side code that lives alongside the frontend in the same project).

In this project Next.js provides:

- The `/` (dashboard) and `/settings` pages via the `app/` directory
- The `/api/emergency-call` endpoint that runs server-side to call Twilio
- The development server (`npm run dev`) that rebuilds code live

What is Tailwind CSS?

Tailwind is a utility-first CSS library. Instead of writing custom CSS files, you apply small pre-built classes directly in JSX (`text-red-400` , `flex` , `p-4` , etc.). This keeps all styling co-located with the markup and makes it easy to make things look consistent.

What are shadcn/ui components?

shadcn/ui is a library of pre-built, accessible UI components (buttons, cards, badges, separators, etc.) styled with Tailwind. In this project they are used as-is for layout and minor UI elements. They require no custom code — just import and use.

---

3. Hardware & Sensors

Sensor	What it measures	Interface
VL53L1X (ToF laser rangefinder)	Distance to object on the left side (blind spot)	I²C
MPU-6050 (IMU)	Acceleration (X/Y/Z) and rotation (gyro X/Y/Z)	I²C
MAX32664 / SparkFun Bio Sensor Hub	Heart rate (BPM) and blood oxygen (SpO_2)	I²C
MAX9814 (microphone)	Ambient sound level (peak amplitude)	Analog (ADC)
LED	Visual indicator when blind spot is active	GPIO output

All I²C sensors share the same two wires (SDA = GPIO 21, SCL = GPIO 22 on the ESP32). The microcontroller is an **ESP32** running Arduino firmware.

Pin Map

```
MIC_PIN   = GPIO 39  (analog input, MAX9814 OUT)
MPU_INT   = GPIO 13  (not used for interrupt, just reserved)
RESET_PIN = GPIO 14  (MAX32664 reset)
MFIO_PIN  = GPIO  2  (MAX32664 mode/FIFO select)
LED_PIN   = GPIO  4  (blind spot indicator LED)
```

---

4. Arduino Code Explained

The Arduino sketch lives at `arduino/main/main.ino`. It does three things: initialize the sensors, read them in a loop, and print a JSON line over USB serial every 200 ms.

Setup Phase

```
void setup() {
  Serial.begin(115200);   // open USB serial at 115,200 baud
  Wire.begin();           // start I²C on default pins (SDA=21, SCL=22)

  // VL53L1X — laser distance sensor
  tof.setTimeout(500);
  tof.init();                          // initialize over I²C
  tof.setDistanceMode(VL53L1X::Long); // up to ~4 m range
  tof.startContinuous(100);           // take a new reading every 100 ms

  // MPU-6050 — accelerometer + gyroscope
  mpu.begin();
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);  // ±8 g full scale
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);       // ±500 °/s
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);    // low-pass filter

  // MAX32664 — pulse oximeter
  bioHub.begin(Wire, 0x55);   // I²C address 0x55
  bioHub.configBpm(MODE_ONE); // continuous heart rate + SpO₂ mode
  delay(4000);                // sensor needs ~4 s warm-up time

  pinMode(MIC_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
}
```

In plain English: The ESP32 wakes up, opens the USB serial port, and one by one initializes each sensor by sending configuration commands over I²C. The pulse oximeter needs four seconds to warm up before it produces valid readings.

Main Loop — Sensor Reads

The loop runs continuously but is rate-limited to 200 ms per iteration using a timestamp check:

```
void loop() {
  // Only run if 200 ms have passed since last print
```

```
  if (millis() - lastPrint < PRINT_INTERVAL) return;
  lastPrint = millis();
  // ... read sensors and print JSON
}
```

Distance (VL53L1X)

```
uint16_t dist = tof.read(false);      // read latest ranging result (non-blocking)
bool tofTimeout = tof.timeoutOccurred();
bool blindSpot = !tofTimeout && (dist > 0) && (dist <= BLIND_SPOT_MM);
// blindSpot is true when an object is within 1500 mm
```

Accelerometer / Gyroscope (MPU-6050)

```
sensors_event_t accel, gyro, temp;
mpu.getEvent(&accel, &gyro, &temp);

float ax = accel.acceleration.x;  // m/s²
float ay = accel.acceleration.y;
float az = accel.acceleration.z;
float accelMag = sqrtf(ax*ax + ay*ay + az*az);  // total magnitude
bool fall = (accelMag >= FALL_ACCEL_THRESH);     // spike > 20 m/s²
```

The vector magnitude `sqrtf(ax² + ay² + az²)` captures the total force regardless of which direction the sensor is oriented. At rest this value is approximately **9.8 m/s²** (gravity). A hard impact spikes it well above 20.

Heart Rate & SpO_2 (MAX32664)

```
bioData body = bioHub.readBpm();
// body.heartRate — beats per minute
// body.oxygen    — blood oxygen saturation percentage (0–100)
```

Microphone (MAX9814)

```
int micMin = 4095, micMax = 0;
for (int i = 0; i < MIC_SAMPLES; i++) {    // 64 samples
  int val = analogRead(MIC_PIN);            // 0–4095 (12-bit ADC)
  if (val > micMax) micMax = val;
  if (val < micMin) micMin = val;
  delayMicroseconds(100);                   // ~6.4 ms total window
}
int micPeak = micMax - micMin;  // peak-to-peak amplitude
```

Instead of a single sample, the code takes 64 rapid samples and computes the **peak-to-peak amplitude**. This gives a measure of how loud the environment is regardless of the DC offset of the mic signal.

JSON Output

Every 200 ms the Arduino prints one line that looks like this:

```
{"dist":842,"blind_spot":false,"ax":0.23,"ay":-0.11,"az":9.77,"gx":0.01,"gy":0.00,"gz"
```

Each field:

Field	Type	Meaning
`dist`	integer (mm)	Distance to nearest object. `-1` = sensor timeout
`blind_spot`	boolean	True if object is within 1500 mm
`ax/ay/az`	float (m/s²)	Raw accelerometer axes
`gx/gy/gz`	float (rad/s)	Raw gyroscope axes
`accel_mag`	float (m/s²)	Vector magnitude of acceleration
`fall`	boolean	True if `accel_mag ≥ 20`
`spo2`	integer (%)	Blood oxygen saturation
`bpm`	integer	Heart rate in beats per minute
`mic`	integer (0–4095)	Peak-to-peak microphone amplitude
`ts`	integer (ms)	Milliseconds since ESP32 boot

---

5. How the Browser Talks to the Arduino

Modern browsers expose the **Web Serial API** — a JavaScript interface that lets a web page directly read and write a USB serial port, without any server or driver. This is what makes CycleWatch work entirely in the browser with no Bluetooth pairing or cloud relay.

The `lib/serial.ts` file wraps the raw Web Serial API calls into simple helper functions:

```
// Check if the current browser supports Web Serial
export function isSerialSupported(): boolean {
  return typeof navigator !== 'undefined' && 'serial' in navigator;
}

// Show the browser's port-picker dialog and return the chosen port
export async function requestPort(): Promise<SerialPortLike> {
  return (navigator as any).serial.requestPort();
}

// Open a port at a given baud rate (must match Arduino's Serial.begin rate)
export async function openPort(port: SerialPortLike, baudRate: number):
Promise<void> {
  await port.open({ baudRate });  // baudRate = 115200
}

// Close a port gracefully
export async function closePort(port: SerialPortLike): Promise<void> {
```

```
  try { await port.close(); } catch { /* already closed */ }
}
```

Reading Lines

The most important function is `readLines`. It is an **async generator** — a special JavaScript function that produces values one at a time, each time the `yield` keyword is reached, and can be iterated with `for await...of`.

```
export async function* readLines(
  port: SerialPortLike,
  signal: AbortSignal,       // lets us stop reading from outside
): AsyncGenerator<string> {

  // Pipe the raw bytes through a text decoder
  const decoder = new TextDecoderStream();
  const pipePromise = port.readable.pipeTo(decoder.writable, { signal });
  const reader = decoder.readable.getReader();

  let buffer = '';
  try {
    while (true) {
      if (signal.aborted) break;
      const { value, done } = await reader.read();  // wait for next chunk
      if (done) break;

      buffer += value;
      const lines = buffer.split('\n');   // split on newline
      buffer = lines.pop() ?? '';         // last piece might be incomplete

      for (const line of lines) {
        const trimmed = line.trim();
        if (trimmed) yield trimmed;       // emit each complete line
      }
    }
  } finally {
    reader.releaseLock();
    await pipePromise.catch(() => {});
  }
}
```

Why a generator? The Arduino sends data continuously. Rather than buffering everything in memory and processing in batches, the generator lets the app process each line as soon as it arrives, keeping the UI responsive.

---

6. Data Flow — End to End

```
[Sensors]
   |
   ▼
```

```
[ESP32 Arduino] — reads every 200 ms, prints JSON line to USB
   │  (USB cable)
   ▼
[Browser Web Serial API] — readLines() async generator yields each JSON string
   │
   ▼
[useSerial hook] — manages port state, broadcasts each line to subscribers
   │  (onLine callback subscription)
   ▼
[useSensorData hook] — parses JSON, updates latest/history state, fires alerts
   │
   ▼
[React components] — read state and re-render automatically
   │
   ├─ BlindSpotRadar    (visualizes distance + mic)
   ├─ HRZoneGauge       (shows BPM on a semicircle gauge)
   ├─ MetricTile        (distance, SpO₂, impact tiles)
   └─ CrashModal        (overlay + countdown + Twilio call)
```

---

7. React Hooks (The Logic Layer)

All of the non-display logic lives in custom hooks inside the `hooks/` folder. A custom hook is just a regular TypeScript function whose name starts with `use` and that can call built-in React hooks inside it. Calling the hook inside a component gives that component reactive access to the hook's state.

`useSerial` — Managing the Serial Connection

This hook owns all the serial port logic: connecting, disconnecting, reading, and broadcasting lines to any part of the app that wants them.

State it tracks:

- `status` — `'disconnected' | 'connecting' | 'connected' | 'error'`
- `port` — the currently open serial port object
- `pairedPorts` — list of previously-authorized ports (so the user can reconnect without re-picking)

Key function: `connect()`

```
const connect = useCallback(async () => {
  try {
    const p = await requestPort();   // shows browser port-picker dialog
    await connectExisting(p);        // opens the port and starts reading
  } catch (err) {
    console.error('Port request cancelled or failed:', err);
  }
}, [connectExisting]);
```

Key function: `startReading()` — the streaming loop

```
const startReading = useCallback(async (activePort: SerialPortLike) => {
  const controller = new AbortController();  // used to stop the loop
```

```
  abortRef.current = controller;
  try {
    // readLines yields one JSON string per iteration
    for await (const line of readLines(activePort, controller.signal)) {
      // broadcast to every subscriber (e.g. useSensorData)
      listenersRef.current.forEach(cb => cb(line));
    }
  } catch { /* stream ended */ }
  finally {
    setStatus('disconnected');
    setPort(null);
  }
}, []);
```

Key function: `onLine()` — pub/sub subscription

```
const onLine = useCallback((cb: (line: string) => void) => {
  listenersRef.current.add(cb);         // add this callback to the set
  return () => listenersRef.current.delete(cb);  // return an unsubscribe function
}, []);
```

Any component or hook can call `onLine(myCallback)` and will receive every line. Calling the returned function unsubscribes.

`useSensorData` — Parsing & Alert Logic

This hook subscribes to `onLine`, parses each JSON string, stores the latest packet, maintains a rolling history for charts, and manages timed alert states.

Input: `onLine` function from `useSerial`

Output:

```
{
  latest: SensorPacket | null,   // most recent parsed data
  history: ChartPoint[],         // last 60 readings for charts
  fallAlert: boolean,            // true for 5 s after a fall event
  blindSpotAlert: boolean,       // true for 2 s after a blind spot event
  packetCount: number,           // total packets received
}
```

Parsing each line:

```
const handleLine = useCallback((line: string) => {
  let packet: SensorPacket;
  try {
    packet = JSON.parse(line) as SensorPacket;
  } catch {
    return;  // ignore non-JSON lines (e.g. boot messages like "VL53L1X OK")
  }
```

```
  setLatest(packet);
  setPacketCount(c => c + 1);

  // Add to history, cap at CHART_HISTORY (60) points
  setHistory(prev => {
    const point = { ts: packet.ts, dist: packet.dist === -1 ? 0 : packet.dist, ...
};
    const next = [...prev, point];
    return next.length > CHART_HISTORY ? next.slice(-CHART_HISTORY) : next;
  });

  // Fall alert — stays up for 5 seconds
  if (packet.fall || packet.accel_mag >= FALL_THRESHOLD) {
    setFallAlert(true);
    if (fallTimerRef.current) clearTimeout(fallTimerRef.current);
    fallTimerRef.current = setTimeout(() => setFallAlert(false), 5000);
  }

  // Blind spot alert — stays up for 2 seconds
  if (packet.blind_spot) {
    setBlindSpotAlert(true);
    if (blindTimerRef.current) clearTimeout(blindTimerRef.current);
    blindTimerRef.current = setTimeout(() => setBlindSpotAlert(false), 2000);
  }
}, []);
```

Why `useRef` for timers? `setTimeout` returns an ID you need to cancel if a new alert fires before the old one expires. Storing the ID in a `ref` (not `state`) means updating it does not trigger a re-render. This is a common React pattern for anything that is a side-effect handle rather than UI data.

`useSettings` — Persisted User Preferences

This hook reads and writes user settings (age, emergency contact name, emergency contact phone number) to `localStorage`, so they survive page refreshes.

```
export function useSettings() {
  const [settings, setSettings] = useState<Settings>(DEFAULTS);
  const [loaded, setLoaded] = useState(false);

  // On first mount, load from localStorage
  useEffect(() => {
    try {
      const raw = localStorage.getItem('cyclewatch_settings');
      if (raw) setSettings({ ...DEFAULTS, ...JSON.parse(raw) });
    } catch { /* ignore parse errors */ }
    setLoaded(true);
  }, []);   // empty dependency array → runs only once

  // Merge partial updates and save back to localStorage
  function save(next: Partial<Settings>) {
    const updated = { ...settings, ...next };
```

```
    setSettings(updated);
    localStorage.setItem('cyclewatch_settings', JSON.stringify(updated));
  }

  // Derived value: max heart rate formula (220 - age)
  return { settings, save, loaded, maxHR: 220 - settings.age };
}
```

Usage in a component:

```
const { settings, save, maxHR } = useSettings();
// maxHR is used by HRZoneGauge to calculate zone percentages
// e.g. for a 30-year-old: maxHR = 220 - 30 = 190 bpm
```

---

8. Application State & Context

The Problem: Sharing Serial State

Both the main dashboard page and potential sub-components need access to the serial connection (to check connection status, connect, disconnect). Passing it down through props from parent to child to grandchild is called "prop drilling" and gets messy quickly.

The Solution: React Context

`context/SerialContext.tsx` creates a **Context** — a global store that any component in the app can read without props:

```
const SerialContext = createContext<UseSerialReturn | null>(null);

// Provider: wraps the whole app and holds the single useSerial instance
export function SerialProvider({ children }: { children: React.ReactNode }) {
  const serial = useSerial();     // one instance shared by everyone
  return (
    <SerialContext.Provider value={serial}>
      {children}
    </SerialContext.Provider>
  );
}

// Hook: any component calls this to access serial state/functions
export function useSerialContext(): UseSerialReturn {
  const ctx = useContext(SerialContext);
  if (!ctx) throw new Error('useSerialContext must be used inside SerialProvider');
  return ctx;
}
```

In `app/layout.tsx`, the whole application is wrapped in `<SerialProvider>`. Then anywhere in the app:

```
// In any component, anywhere in the tree:
const serial = useSerialContext();
```

```
serial.connect();          // connect to Arduino
serial.disconnect();       // disconnect
serial.status             // 'connected' | 'disconnected' | 'connecting' | 'error'
serial.onLine(callback)   // subscribe to incoming lines
```

---

9. UI Components

`BlindSpotRadar`

A custom SVG radar visualization that shows the left side of the bike as a semicircle with three colored danger zones. The radar is drawn entirely using SVG `<path>` elements computed from the live distance reading.

How the geometry works:

```
const CX = 370;    // bike position (right edge of radar)
const CY = 180;    // vertical center
const MAX_R = 310; // pixel radius = 1500 mm real world

// Convert mm to pixel X coordinate
function toX(distMm: number) {
  return CX - (distMm / BLIND_SPOT_THRESHOLD_MM) * MAX_R;
}

// SVG path for a left-facing semicircle sector
function sector(r: number) {
  return `M ${CX} ${CY - r} A ${r} ${r} 0 0 0 ${CX} ${CY + r} L ${CX} ${CY} Z`;
}
```

When an object is detected, a dot appears at `toX(dist)` on the center line, and a dashed line connects it to the bike icon.

`HRZoneGauge`

Uses the third-party `react-gauge-component` library to render a semicircle gauge. The arc is divided into six color bands matching standard cycling heart rate zones. The zone is calculated as a percentage of the rider's age-based max heart rate:

```
const pct = (bpm / maxHR) * 100;

const zone =
  pct >= 90 ? { name: 'Z5 Max',       color: '#ef4444' } :
  pct >= 80 ? { name: 'Z4 Threshold', color: '#f97316' } :
  pct >= 70 ? { name: 'Z3 Cardio',    color: '#eab308' } :
  pct >= 60 ? { name: 'Z2 Fat Burn',  color: '#22c55e' } :
  pct >= 50 ? { name: 'Z1 Warm-up',   color: '#3b82f6' } :
              { name: 'Rest',          color: '#52525b' };
```

`MetricTile`

A simple display tile used for Distance, SpO_2, and Impact. It changes the value's text color based on severity thresholds:

```
// Distance color logic
const distColor =
  dist === -1                     ? 'text-zinc-600'   :   // no reading
  dist <= DISTANCE_DANGER_MM      ? 'text-red-400'    :   // < 500 mm
  dist <= DISTANCE_WARNING_MM     ? 'text-orange-400' :   // < 1000 mm
  dist <= BLIND_SPOT_THRESHOLD_MM ? 'text-yellow-400' :   // < 1500 mm
                                    'text-white';          // clear
```

`CrashModal`

A full-screen overlay that appears when a fall/crash is detected. It has two key mechanics:

1. **10-second countdown** — after 10 s without dismissal, the app calls the emergency contact via Twilio
2. **Hold-to-dismiss button** — the rider must hold a button for 2 s to dismiss (prevents accidental taps during a real crash)

```
// Hold-to-dismiss: track how long the button is pressed
const startHold = useCallback(() => {
  holdStart.current = Date.now();
  holdRef.current = setInterval(() => {
    const progress = Math.min(100, ((Date.now() - holdStart.current) / 2000) * 100);
    setHoldProgress(progress);
    if (progress >= 100) {
      clearInterval(holdRef.current!);
      onDismiss();  // only dismiss after 2 full seconds
    }
  }, 30);   // check every 30 ms for smooth animation
}, [onDismiss]);
```

---

10. The Emergency Call API

When the crash modal countdown hits zero, the frontend sends a POST request to `/api/emergency-call`. This is a **Next.js API Route** — server-side code that runs on Node.js, not in the browser. This is necessary because Twilio API credentials cannot be safely stored in the browser.

Frontend trigger (in CrashModal):

```
fetch('/api/emergency-call', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    to: emergencyPhone,      // e.g. "+15551234567"
    contactName: emergencyName,
  }),
});
```

Server handler (`app/api/emergency-call/route.ts`):

```
export async function POST(req: Request) {
  const { to, contactName } = await req.json();

  // Credentials from environment variables (never exposed to browser)
  const accountSid = process.env.TWILIO_ACCOUNT_SID;
  const authToken  = process.env.TWILIO_AUTH_TOKEN;
  const from       = process.env.TWILIO_FROM_NUMBER;

  // Sanitize the name to prevent injection in TwiML
  const name = contactName ? contactName.replace(/[<>&"]/g, '') : 'the rider';

  // TwiML: XML that tells Twilio what to say during the call
  const twiml = `<Response>
    <Say voice="alice" language="en-US">
      This is an automated emergency alert from Cycle Watch.
      ${name} has been detected in a possible crash...
    </Say>
  </Response>`;

  // Make the actual phone call via Twilio SDK
  const client = twilio(accountSid, authToken);
  await client.calls.create({ twiml, to, from });

  return NextResponse.json({ success: true });
}
```

How Twilio works: Twilio is a cloud service that can make phone calls programmatically. The app provides TwiML (Twilio Markup Language), which is XML that describes what should be spoken using text-to-speech during the call. Twilio dials the number, and when the person answers, it reads the message using its "Alice" voice.

---

11. Settings & Local Storage

The Settings page (`/settings`) lets the rider enter:

- **Age** — used to compute max heart rate (220 – age)
- **Emergency Contact Name** — shown in the crash modal and spoken in the Twilio call
- **Emergency Contact Phone** — the number Twilio dials

The phone number is automatically normalized to E.164 format (`+1XXXXXXXXXX`) because that is what Twilio requires:

```
function handlePhoneChange(raw: string) {
  let digits = raw.replace(/[^\d]/g, '');  // strip all non-digits
  if (digits.startsWith('1') && digits.length > 1) {
    digits = digits.slice(1);  // remove leading country code if user typed it
  }
```

```
  setPhone(digits ? `+1${digits}` : '');   // always store as +1...
}
```

All settings are stored in `localStorage` under the key `cyclewatch_settings` as a JSON string. The `useSettings` hook loads them on first render and provides a `save()` function to update and persist them.

---

12. Constants & Thresholds

All magic numbers live in `lib/constants.ts` so they can be tuned in one place and are shared between the Arduino code comments and the frontend logic:

```
export const BAUD_RATE             = 115200;   // Serial speed (must match Arduino)
export const BLIND_SPOT_THRESHOLD_MM = 1500;   // 1.5 m — blind spot zone boundary
export const DISTANCE_DANGER_MM    = 500;      // < 50 cm = danger (red)
export const DISTANCE_WARNING_MM   = 1000;     // < 1 m  = warning (orange)
export const FALL_THRESHOLD        = 20;       // m/s² — impact event threshold
export const SPO2_LOW              = 95;       // below 95% SpO₂ is concerning
export const BPM_LOW               = 50;       // resting below 50 bpm is unusual
export const BPM_HIGH              = 160;      // high-intensity threshold
export const MIC_LOUD_THRESHOLD    = 800;      // ADC peak-to-peak for "loud"
environment
export const CHART_HISTORY         = 60;       // keep last 60 data points for
graphs
```

---

13. Key Code Examples

Example 1: Connecting to the Arduino

```
// In any component:
const serial = useSerialContext();

// When the user clicks "Connect Device":
<button onClick={serial.connect}>Connect Device</button>

// What happens inside serial.connect():
// 1. Browser shows a native port-picker dialog
// 2. User selects the ESP32's COM port
// 3. Port is opened at 115200 baud
// 4. Background loop starts reading lines and broadcasting them
```

Example 2: Reading Live Sensor Values

```
// In the main page component:
const serial = useSerialContext();
const { latest, fallAlert, blindSpotAlert } = useSensorData(serial.onLine);

// `latest` is the most recently parsed SensorPacket, e.g.:
```

```
// latest.dist      → 842 (mm)
// latest.bpm       → 72 (beats per minute)
// latest.spo2      → 98 (percent)
// latest.accel_mag → 9.78 (m/s², near gravity = no crash)
// latest.fall      → false
// latest.blind_spot → false
```

Example 3: Conditional Color Coding

```
// Color changes automatically based on sensor value
const spo2Color =
  spo2 === 0      ? 'text-zinc-600'   :  // no reading yet — grey
  spo2 < 95       ? 'text-red-400'    :  // dangerously low — red
  spo2 < 97       ? 'text-yellow-400' :  // slightly low — yellow
                    'text-white';         // normal — white

<MetricTile
  label="Blood Oxygen"
  value={spo2 === 0 ? '—' : spo2}
  unit="%"
  valueColor={spo2Color}   // Tailwind class applied to the number
/>
```

Example 4: The Alert State Machine

```
// Fall alert lifecycle:
// 1. Arduino reads accel_mag >= 20 → sets fall: true in JSON
// 2. useSensorData parses packet → setFallAlert(true)
// 3. A 5-second timer starts → after 5 s, setFallAlert(false)
// 4. Page component detects fallAlert changed to true:
useEffect(() => {
  if (fallAlert) setCrashOpen(true);  // open the crash modal
}, [fallAlert]);
// 5. CrashModal shows and starts its own 10-second countdown
// 6. If not dismissed → fetch('/api/emergency-call') → Twilio calls contact
```

Example 5: Persistent Last-Known Value

```
// The pulse oximeter takes time to produce a reading.
// Without this pattern, bpm would show 0 every time the sensor sends 0.
const [lastBpm, setLastBpm] = useState(0);

useEffect(() => {
  if ((latest?.bpm ?? 0) > 0) setLastBpm(latest!.bpm);
}, [latest?.bpm]);

// Now use lastBpm instead of latest.bpm in the UI.
```

```
// The displayed value "sticks" at the last valid reading.
const bpm = lastBpm;
```

Example 6: SVG Radar Zone Path

```
// The radar zones are drawn as SVG arc sectors.
// This function creates the SVG path data for a left-facing semicircle.
function sector(r: number) {
  // M = move to start point (top of the arc)
  // A = arc: radius rx, radius ry, rotation, large-arc-flag, sweep-flag, end-x,
end-y
  // L = line to center
  // Z = close path
  return `M ${CX} ${CY - r} A ${r} ${r} 0 0 0 ${CX} ${CY + r} L ${CX} ${CY} Z`;
}

// Three zones rendered on top of each other (largest first):
<path d={sector(MAX_R)}    fill="rgba(234,179,8,0.04)"  />  // yellow zone (1.5 m)
<path d={sector(warningR)} fill="rgba(249,115,22,0.07)" />  // orange zone (1 m)
<path d={sector(dangerR)}  fill="rgba(239,68,68,0.10)"  />  // red zone (500 mm)
```

End of documentation — CycleWatch, CS 479 Final Project